

Практичне завдання 2: Логістична регресія та федеративне навчання

Мета завдання

Зрозуміти архітектуру багатокласової логістичної регресії, навчитися самостійно реалізовувати модель засобами `pymru`, зрозуміти принципи федеративного навчання, правильно розподілити дані між клієнтами та сервером, реалізувати агрегацію ваг та проаналізувати базові ефекти розподіленого навчання.

1 Рекомендований датасет

Для виконання роботи рекомендується використати датасет **Dry Bean Dataset** з Kaggle.

Джерело: <https://www.kaggle.com/datasets/muratkokludataset/dry-bean-dataset>

Характеристика датасету:

- Цільова змінна: `Class` — назва сорту квасолі.
- Кількість класів: 7 (багатокласова класифікація).
- Ознаки: числові фізичні характеристики зерен (площа, периметр, довжина, ширина тощо).

Також дозволяється використати інший датасет з Kaggle за умови, що:

- задача є багатокласовою класифікацією (не менше 3 класів);
- датасет має достатню кількість прикладів для розподілу між кількома клієнтами;
- студент може пояснити, чому обраний датасет підходить для федеративного навчання.

Приклади альтернативних датасетів: Fashion-MNIST, Forest Cover Type, Human Activity Recognition, будь-який табличний датасет із кількома класами.

2 Етапи виконання

2.1 Етап 1. Дослідження даних та попередня обробка

1. Первинний аналіз даних:

- структура датасету та типи ознак;
- пропущені значення;

- розподіл цільової змінної;
- основні статистики числових ознак;
- кореляції між ознаками.

2. Відбір ознак:

- визначте, які ознаки доцільно використовувати для моделі;
- проведіть аналіз кореляцій між ознаками та цільовою змінною;
- зверніть увагу на сильні кореляції між самими ознаками (мультиколінеарність).

Важливо: відбір ознак має виконуватися без використання тестової вибірки.

3. Розбиття вибірки.

Спочатку сформууйте три множини:

- **Global Test** — 15–20% даних, стратифікована вибірка за класами. Ця частина зберігається на сервері та використовується лише для фінальної оцінки якості моделі.
- **Global Validation** — 10–15% даних, стратифікована вибірка за класами. Використовується для підбору гіперпараметрів та порівняння моделей. Не передається клієнтам.
- **FL Train Pool** — решта даних. Саме ця частина буде розподілена між клієнтами у федеративному навчанні.

Питання: чому розбиття даних потрібно виконувати до масштабування, кодування або інших операцій, які навчаються на даних? Поясніть термін *Data Leakage*.

4. Кодування категоріальних ознак.

Якщо обраний датасет має категоріальні ознаки, оберіть і обґрунтуйте спосіб кодування:

- One-Hot Encoding;
- Ordinal Encoding;
- Target Encoding.

5. Масштабування.

Застосуйте стандартизацію або нормалізацію для числових ознак.

Параметри масштабування мають бути обчислені лише на тренувальних даних. Потім ці самі параметри застосовуються до валідаційної та тестової множин.

У федеративному режимі можливі два коректні підходи:

- Спрощений: скалер навчається на **FL Train Pool** до розподілу даних між клієнтами. Це допустимо, якщо в роботі явно зазначено, що такий крок є частиною симуляції.
- Більш федеративний: кожен клієнт обчислює локальні середні та дисперсії, а сервер агрегує ці статистики зважено за кількістю прикладів. Потім агреговані статистики застосовуються до всіх клієнтів, валідації та тесту.

Важливо: тестова вибірка не повинна використовуватися для обчислення статистик масштабування.

2.2 Етап 2. Реалізація багатокласової логістичної регресії засобами numpy

Реалізуйте клас або набір функцій багатокласової логістичної регресії переважно засобами numpy.

Нехай:

$$X \in \mathbb{R}^{m \times d}$$

— матриця об'єктів, де m — кількість прикладів, d — кількість ознак.

Нехай:

$$Y \in \{0, 1\}^{m \times C}$$

— one-hot кодування цільової змінної, де C — кількість класів.

Параметри моделі:

$$W \in \mathbb{R}^{d \times C}, \quad b \in \mathbb{R}^{1 \times C}.$$

Лінійна частина моделі:

$$Z = XW + b.$$

Ймовірності класів отримуються за допомогою softmax:

$$P = \text{softmax}(Z),$$

де для кожного рядка:

$$P_{ic} = \frac{\exp(Z_{ic})}{\sum_{j=1}^C \exp(Z_{ij})}.$$

Для числової стабільності реалізуйте softmax із відніманням максимуму рядка.

Функція втрат без регуляризації:

$$J(W, b) = -\frac{1}{m} \sum_{i=1}^m \sum_{c=1}^C Y_{ic} \log P_{ic}.$$

Із Ridge-регуляризацією:

$$J(W, b) = -\frac{1}{m} \sum_{i=1}^m \sum_{c=1}^C Y_{ic} \log P_{ic} + \frac{\lambda}{2} \|W\|_F^2.$$

Параметр b зазвичай не регуляризується.

Для Ridge-регуляризації градієнти мають вигляд:

$$\frac{\partial J}{\partial W} = \frac{1}{m} X^\top (P - Y) + \lambda W,$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \mathbf{1}_m^\top (P - Y).$$

У межах реалізації потрібно створити:

- функцію `softmax`;
- функцію обчислення ймовірностей `predict_proba`;

- функцію передбачення класу `predict`;
- функцію втрат `loss`;
- функцію обчислення градієнтів;
- функцію оновлення параметрів моделі;
- навчання за допомогою міні-батчевого градієнтного спуску.

Використання циклів `for` для обчислення функції втрат або градієнтів за прикладами заборонено. Використовуйте векторизацію та матричні операції `numpy`.

Цикли по епохах, ітераціях, клієнтах або батчах дозволені.

2.3 Етап 3. Центральна навчана базова модель

Навчіть логістичну регресію на централізованих даних, тобто на всьому `FL Train Pool`.

Ця модель буде використовуватися як базова лінія для порівняння з федеративними моделями.

Під час навчання:

- підберіть швидкість навчання;
- оберіть розмір батчу;
- дослідіть вплив Ridge-регуляризації;
- використовуйте `Global Validation` для вибору гіперпараметрів;
- фінальну оцінку виконайте лише на `Global Test`.

Обчисліть метрики:

- accuracy;
- macro-F1;
- balanced accuracy;
- log-loss;
- confusion matrix.

2.4 Етап 4. Розподіл даних між клієнтами та сервером

Це ключовий етап роботи. Правильний розподіл даних є критично важливим для методичної коректності федеративного навчання.

2.4.1 Архітектура системи

У федеративному навчанні є два типи учасників:

- **Сервер (агрегатор)** — центральний вузол, який координує навчання, агрегує параметри моделей та оцінює глобальну модель.
- **Клієнти** — учасники, які мають власні локальні дані та навчають локальні моделі.

2.4.2 Принципи розподілу даних

1. Сервер має доступ тільки до:

- **Global Test** — для фінальної оцінки якості глобальної моделі;
- **Global Validation** — для моніторингу прогресу навчання та підбору гіперпараметрів;
- параметрів моделей (ваг), які повертають клієнти.

Сервер НЕ має доступу до:

- сирих даних клієнтів;
- локальних тренувальних вибірок.

2. Кожен клієнт має доступ тільки до:

- власної локальної тренувальної вибірки D_k ;
- (опціонально) власної локальної валідаційної вибірки, сформованої з D_k .

Клієнт НЕ має доступу до:

- **Global Test**;
- **Global Validation**;
- даних інших клієнтів;
- параметрів моделей інших клієнтів (лише агреговану глобальну модель від сервера).

2.4.3 Методика розподілу FL Train Pool між клієнтами

Нехай K — кількість клієнтів. Позначимо:

- D_k — локальний набір тренувальних даних клієнта k ;
- $n_k = |D_k|$ — кількість тренувальних прикладів клієнта k ;
- $\sum_{k=1}^K n_k = |\text{FL Train Pool}|$.

Вимоги до розподілу:

1. Кожен приклад з **FL Train Pool** має належати рівно одному клієнту.
2. Розподіл має бути задокументований: кількість клієнтів, кількість прикладів у кожного клієнта, розподіл класів у кожного клієнта.
3. Розподіл має бути відтворюваним (використовуйте фіксоване випадкове зерно).

2.4.4 Типи розподілу даних

Рекомендується дослідити два типи розподілу:

1. IID-розподіл (Independent and Identically Distributed)

Кожен клієнт отримує приблизно однакову кількість прикладів, а розподіл класів у кожного клієнта приблизно відповідає глобальному розподілу.

Як реалізувати:

1. Перемішайте всі приклади з **FL Train Pool**.
2. Розділіть їх на K приблизно рівних частин.
3. Або використайте стратифіковане розбиття, щоб зберегти пропорції класів.

Це базовий сценарій, у якому федеративне навчання зазвичай працює стабільно.

2. Non-IID розподіл за допомогою розподілу Діріхле

Для моделювання неоднорідних даних використайте розподіл Діріхле.

Як реалізувати:

1. Для кожного класу c згенеруйте пропорції розподілу прикладів цього класу між клієнтами:

$$(p_{1c}, p_{2c}, \dots, p_{Kc}) \sim \text{Dirichlet}(\alpha, \alpha, \dots, \alpha).$$

2. Для кожного класу c розподіліть усі приклади цього класу між клієнтами відповідно до отриманих пропорцій p_{kc} .

Параметр α контролює рівень неоднорідності:

- велике α (наприклад, 5 або 10) дає розподіл, близький до IID;
- $\alpha = 1$ дає помірну неоднорідність;
- $\alpha = 0.3$ або $\alpha = 0.1$ дає сильну неоднорідність;
- при дуже малому α окремі клієнти можуть майже не мати певних класів.

Вимоги до візуалізації розподілу:

- побудуйте теплову карту або стовпчикову діаграму розподілу класів між клієнтами;
- покажіть кількість прикладів у кожного клієнта;
- перевірте, чи немає клієнтів із занадто малою кількістю прикладів (наприклад, менше 10);
- поясніть, як нерівномірний розподіл класів впливає на якість глобальної моделі.

2.4.5 Поширені помилки при розподілі даних

НЕПРАВИЛЬНО:

- Використовувати **Global Test** або **Global Validation** для локального навчання клієнтів.
- Дозволяти клієнтам бачити дані інших клієнтів.
- Масштабувати дані до розбиття на множини (**Data Leakage**).
- Не документувати розподіл даних.

ПРАВИЛЬНО:

- Спочатку розбити дані на **Global Test**, **Global Validation** та **FL Train Pool**.
- Масштабувати дані після розбиття.
- Розподілити лише **FL Train Pool** між клієнтами.
- Задokumentувати розподіл та візуалізувати його.

2.5 Етап 5. Симуляція федеративного навчання

Реалізуйте симуляцію федеративного навчання на одному комп'ютері.

2.5.1 Алгоритм федеративного навчання (FedAvg)

1. Сервер ініціалізує глобальну модель w^0 .
2. На кожному раунді t сервер обирає підмножину клієнтів S_t (або всіх клієнтів).
3. Сервер передає глобальну модель w^t обраним клієнтам.
4. Кожен клієнт $k \in S_t$ навчає локальну модель протягом E локальних епох, починаючи з w^t .
5. Клієнт повертає оновлені параметри w_k^{t+1} серверу.
6. Сервер агрегує параметри та формує нову глобальну модель w^{t+1} .
7. Глобальна модель оцінюється на **Global Validation**.

2.5.2 Агрегація wag

Агрегація **FedAvg** виконується як зважене середнє параметрів:

$$w^{t+1} = \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} w_k^{t+1}.$$

Тобто клієнти з більшою кількістю тренувальних прикладів мають більший вплив на глобальну модель.

Параметри w включають матрицю ваг W та вектор зміщень b .

Питання: чому ваги клієнтів доцільно зважувати за кількістю прикладів? Що станеться, якщо використовувати просте середнє без зважування?

2.6 Етап 6. Дослідження ефектів федеративного навчання

Потрібно порівняти три типи моделей:

1. Локальні клієнтські моделі — моделі, навчені лише на даних окремих клієнтів.
2. Глобальна серверна модель — модель, отримана після агрегації ваг.
3. Централізована модель — модель, навчена на всіх тренувальних даних (FL Train Pool).

Для порівняння побудуйте таблицю з метриками на **Global Test**: accuracy, macro-F1, balanced accuracy, log-loss.

2.6.1 Обов'язкові експерименти

Експеримент 1. Вплив неоднорідності даних

Порівняйте результати для різних значень параметра Діріхле:

- $\alpha = 5$ (близько до IID);
- $\alpha = 1$ (помірна неоднорідність);
- $\alpha = 0.3$ (сильна неоднорідність).

Побудуйте графік залежності фінальної якості глобальної моделі від α . Поясніть:

- чому Non-IID дані можуть погіршувати федеративне навчання;
- чи може глобальна модель бути гіршою за централізовану;
- чи може глобальна модель бути гіршою за деякі локальні моделі.

Експеримент 2. Вплив кількості локальних епох

Порівняйте:

- $E = 1$;
- $E = 5$;
- $E = 10$.

Поясніть, що таке *client drift* (клієнтський дрейф), тобто надмірне відхилення локальних моделей від глобальної моделі, і чому велика кількість локальних епох може погіршувати глобальну модель у Non-IID сценарії.

2.7 Етап 7. Додатковий експеримент (опційно)

Якщо часу достатньо, виконайте один із наведених експериментів:

Експеримент А. Вплив кількості клієнтів

Порівняйте результати для $K = 3$, $K = 5$, $K = 10$ клієнтів.

Поясніть, як змінюється якість глобальної моделі зі збільшенням кількості клієнтів.

Експеримент Б. Персоналізація

Після отримання глобальної моделі кожен клієнт донавчає її локально протягом 1–2 епох на своїх даних.

Порівняйте якість персоналізованих моделей з глобальною моделлю на локальних тестових вибірках клієнтів.

3 Теоретичні питання

Під час захисту роботи потрібно відповісти на такі питання:

1. Чому для багатокласової класифікації використовують softmax?
2. Чому крос-ентропія є природною функцією втрат для логістичної регресії?
3. Чим відрізняється Ridge-регуляризація від Lasso-регуляризації?
4. Чому параметр зміщення b зазвичай не регуляризують?
5. Що таке Data Leakage?
6. Чому тестову вибірку не можна використовувати для підбору гіперпараметрів?
7. Чому параметри масштабування потрібно навчати лише на тренувальних даних?
8. Що таке федеративне навчання і які його основні переваги?
9. Чому сервер не повинен отримувати сирі дані клієнтів?
10. Чому агрегація ваг за методом FedAvg використовує зважування за кількістю прикладів?
11. Що станеться, якщо агрегувати параметри клієнтів без зважування?
12. Що таке Non-IID дані?
13. Чому федеративне навчання може погіршуватися на Non-IID даних?
14. Що таке клієнтський дрейф (client drift)?
15. Як кількість локальних епох впливає на глобальну модель?
16. Чи може глобальна модель бути гіршою за централізовану модель? Чому?
17. Які метрики краще використовувати для багатокласової класифікації з можливим дисбалансом класів?

4 Вимоги до звіту

Звіт має містити не лише код і графіки, а й пояснення.

Для кожного графіка або таблиці потрібно написати висновок:

- що показано;
- яка закономірність спостерігається;
- чому так відбувається;
- як це впливає на вибір моделі або гіперпараметрів.

Обов'язково включіть:

- результати EDA;

- обґрунтування вибору датасету;
- схему розбиття даних;
- схему розподілу даних між клієнтами;
- візуалізацію розподілу класів між клієнтами;
- графіки навчання централізованої моделі;
- графіки навчання федеративної моделі за раундами;
- таблицю порівняння локальних, глобальної та централізованої моделей;
- аналіз впливу неоднорідності даних;
- аналіз впливу кількості локальних епох;
- відповіді на теоретичні питання;
- загальний висновок.

5 Формат здачі

Роботу потрібно подати у вигляді Jupyter Notebook формату `.ipynb`, завантаженого на GitHub.

Ноутбук має містити:

- чистий і прокоментований код;
- Markdown-комірки з теоретичними поясненнями та висновками;
- результати EDA;
- реалізацію логістичної регресії на `pymru`;
- реалізацію федеративного навчання;
- реалізацію агрегації ваг;
- графіки розподілу даних між клієнтами;
- графіки навчання моделей;
- таблиці метрик;
- відповіді на теоретичні питання;
- загальний висновок.

Код повинен бути структурований. Модель, функцію втрат, градієнти та федеративну агрегацію потрібно реалізувати самостійно засобами `pymru`.

Дозволяється використовувати:

- `pandas` для завантаження та обробки даних;
- `pymru` для моделі та навчання;

- `matplotlib` або `seaborn` для візуалізації;
- `scikit-learn` для метрик, стратифікованого розбиття та допоміжних перетворень.

Використання готових моделей типу `LogisticRegression` або `SGDClassifier` для отримання основних результатів заборонено. Їх можна використовувати лише для допоміжної перевірки правильності градієнтів або як додатковий орієнтир, якщо це окремо зазначено у звіті.